Software Engineering Excellence
Taxi Tap by Git It Done

Contents

1 Introduction

TaxiTap represents a comprehensive mobile platform designed to revolutionize South Africa's minibus taxi industry. Our team, Git It Done, has developed a modern solution that bridges traditional taxi operations with cutting-edge technology, creating a seamless experience for both passengers and operators. This application demonstrates our commitment to software engineering excellence across three critical dimensions: how we build software, how we operate, and how we collaborate.

Our solution addresses real-world transportation challenges while maintaining the flexible, multi-passenger nature that makes taxis essential to South African urban mobility. Through rigorous engineering practices, comprehensive testing, and collaborative development processes, we have created a robust, scalable, and user-friendly platform that exemplifies software engineering excellence.

2 How We Build Software

2.1 Coding Standards

TaxiTap demonstrates engineering excellence through comprehensive coding standards and practices that ensure code quality, maintainability, and consistency across our entire development team.

2.1.1 Language and Framework Standards

Our project employs a modern, type-safe technology stack:

- **TypeScript/JavaScript**: Primary programming language ensuring type safety and better developer experience

- **React Native**: Cross-platform mobile development framework
- **Convex**: Serverless backend with real-time capabilities

2.1.2 Naming Conventions

We maintain strict naming conventions across all code:

- **Files and directories**: camelCase and PascalCase for clear identification
- **Variables**: camelCase for consistency
- **Constants**: UPPER_SNAKE_CASE and camelCase
- **Classes/Components**: PascalCase for React components
- **Functions**: camelCase for all function declarations

2.1.3 Code Style Guidelines

Our code style is enforced through automated tooling:

- **Indentation**: 2 spaces for consistent formatting
- **Maximum line length**: 100 characters for readability
- **Semicolons**: Required at the end of statements
- **String quotes**: Single quotes for consistency
- **Block structure**: Always use curly braces for code blocks

2.2 Code Reviews

Our code review process ensures quality and knowledge sharing:

2.2.1 Automated Quality Gates

- **ESLint Integration**: Comprehensive linting rules covering TypeScript, React Native, and React best practices
- **Prettier Formatting**: Automated code formatting ensures consistent style across all files
- **TypeScript Compilation**: Strict type checking prevents runtime errors
- **GitHub Actions CI/CD**: Automated testing and quality checks on every pull request

2.2.2 Peer Review Process

- **Mandatory Reviews**: All code changes require peer review before merging
- **Clear Commit Messages**: Descriptive commit messages following conventional commit standards
- **Test Coverage Requirements**: New code must include corresponding tests
- **Branch Protection**: Main branch protection prevents direct pushes

2.3 Development Patterns

2.3.1 Architectural Patterns

We implement several key architectural patterns:

Event-Driven Architecture (EDA):

- Real-time location updates and ride requests
- Asynchronous processing for notifications and analytics
- Loose coupling between components
- Scalable event handling for high user loads

Feature-Driven Development (FDD):

- Modular decomposition by functional systems (User System, Vehicle System, Trip System)
- Parallel development capabilities
- Independent feature scaling
- Reduced risk through isolated changes

2.3.2 Repository Structure

Our organized repository structure promotes maintainability:

```
project-root/
 .github/                    % Workflow files
    workflows/
        platform.yml
 assets/                     % All project images
 docs/                       % Comprehensive documentation
 platform/                   % Main application
    app/                    % Frontend components
    convex/                 % Backend functions
    tests/                  % Unit and integration tests
    .eslint.config.mjs      % Linting configuration
    package.json            % Dependencies and scripts
 testing/                    % Performance and security tests
```

3 How We Operate

3.1 Planning

3.1.1 Project Management

Our planning demonstrates operational excellence through structured project management:

GitHub Project Board Integration:

- **Transparent Workflow**: Public project board accessible at `https://github.com/orgs/COS301-SE-2025/projects/265/views/1`
- **Task Tracking**: Comprehensive issue and milestone management
- **Progress Visibility**: Real-time project status updates
- **Team Coordination**: Clear assignment and deadline management

GitFlow Branching Strategy:

- **Production Branch**: Stable main branch for production releases
- **Development Branch**: Integration branch for feature development
- **Feature Branches**: Isolated development for new features
- **Hotfix Branches**: Rapid response to critical issues
- **Release Branches**: Controlled release preparation

3.1.2 Documentation Standards

Comprehensive documentation supports our operational excellence:

- **Technical Documentation**: Architecture, API, and system design documents
- **User Documentation**: Complete user manuals and installation guides
- **Development Guides**: Coding standards, testing policies, and contribution guidelines
- **Version Control**: All documentation maintained in version control with change tracking

3.2 Monitoring

3.2.1 Performance Monitoring

Our monitoring systems ensure optimal system performance:

Comprehensive Testing Framework:

- **Unit and Integration Testing**: 80% minimum code coverage using Jest
- **Performance Testing**: K6 load testing with realistic traffic patterns

- **Stress Testing**: System behavior under extreme load conditions

Performance Metrics:

- **Response Time**: P95 response time less than 3 seconds for all API endpoints
- **Throughput**: Minimum 100 requests per second sustained capacity

3.2.2 Continuous Integration/Continuous Deployment

Automated CI/CD pipeline ensures reliable operations:

GitHub Actions Workflow:

- **Automated Testing**: Frontend and backend tests on every commit
- **Code Quality Checks**: ESLint and Prettier validation
- **Type Checking**: TypeScript compilation verification
- **Deployment Automation**: Automated deployment to Convex cloud
- **Build and Release**: Automated APK generation and GitHub releases

3.3 Data Integrity

3.3.1 Database Design and Management

Our data integrity practices ensure reliable and consistent data:

Schema-Defined Collections:

- **Structured Data Model**: Well-defined collections for users, rides, taxis, and routes
- **Relational Support**: Proper relationships via v.id() references
- **Data Validation**: Server-side validation for all data inputs
- **Type Safety**: TypeScript interfaces ensure compile-time data validation

Real-Time Data Synchronization:

- **Convex Database**: Document-oriented database with real-time sync
- **Automatic Indexing**: Optimized queries with withIndex() for performance
- **Data Consistency**: ACID compliance through Convex's transactional model

3.3.2 Security and Data Protection

Comprehensive security measures protect user data:

Security Testing Framework:

- **Automated Security Scanning**: Python-based security testing suite
- **Vulnerability Assessment**: Regular penetration testing and security audits
- **Authentication Security**: Secure authentication mechanisms with token validation
- **Input Validation**: Protection against SQL injection, XSS, and parameter manipulation

Data Protection Measures:

- **Encryption in Transit**: TLS encryption for all data transmission
- **Access Control**: Role-based access control (RBAC) for different user types
- **Secure Communication**: All API communications encrypted by default

4 How We Collaborate

4.1 Team Values

4.1.1 Collaborative Development Culture

Our team embodies collaborative values that drive engineering excellence:

Knowledge Sharing:

- **Cross-Functional Expertise**: Team members contribute across frontend, backend, and full-stack development, and help each other when necessary or needed
- **Documentation Culture**: Comprehensive documentation of decisions, processes, and development done through our minutes and documentation.
- **Code Reviews**: Peer review process fosters quality improvements and we ensured that we always review each others code
- **Open Communication**: Transparent communication channels for technical discussions

Continuous Learning:

- **Problem-Solving Mindset**: Collaborative approach to tackling complex technical challenges
- **Adaptability**: As team members we have learned to adapt to diverse development environments and requirements, and even changes within our team structure
- **Innovation Focus**: Emphasis on practical solutions and continuous improvement

4.1.2 Quality-First Approach

Our team values prioritize quality and excellence:

Technical Excellence:

- **Type Safety**: TypeScript adoption ensures robust, maintainable code
- **Testing Culture**: Comprehensive testing at unit, integration, and system levels
- **Performance Optimization**: Continuous focus on system performance and scalability
- **Security Awareness**: Proactive security measures and regular security assessments

User-Centric Development:

- **Accessibility Focus**: Applications designed for diverse user literacy levels
- **User Experience**: Intuitive interfaces with responsive design principles
- **Real-World Impact**: Solutions addressing genuine transportation challenges within the Taxi industry
- **Quality Assurance**: Rigorous testing to ensure reliable user experience

4.2 Team Structure

4.2.1 Role-Based Organization

Our team structure optimizes for both specialization and collaboration:

Project Manager - Ann-Marí Oberholzer:

- **Leadership Experience**: Proven track record in team leadership and project coordination
- **Technical Foundation**: Strong background in Java, C++, and web technologies
- **Communication Skills**: Enhanced teamwork and communication abilities
- **Adaptability**: Experience in diverse group settings and project environments

Backend Engineer - Unathi Dlamini:

- **Technical Expertise**: Strong programming and web design skills
- **Collaborative Approach**: Team-oriented problem-solving and continuous learning
- **Technology Exploration**: Active engagement with cybersecurity and software engineering
- **Problem-Solving**: Strong analytical and technical problem-solving abilities

Frontend Engineer - Moyahabo Hamese:

- **User Experience Focus**: Passionate about creating positive, intuitive user experiences
- **Innovation Mindset**: Balance of technical skills, creativity, and problem-solving
- **Community Engagement**: Experience with educational and consulting projects
- **Technical Skills**: Frontend development expertise with cybersecurity awareness

Fullstack/Data Engineer - Nevan Rahman:

- **Leadership Experience**: Senior Team Lead experience in university IT Lab
- **Technical Breadth**: Expertise in frontend, backend, API integration, and data science
- **Communication Excellence**: Exceptional communication skills and team-oriented work ethic
- **Architecture Focus**: Passion for software architecture and full-stack development

4.2.2 Collaborative Workflow

Our team structure promotes effective collaboration:

Parallel Development:

- **Feature-Based Development**: Independent feature development with clear interfaces
- **Cross-Team Collaboration**: Regular communication between all team members
- **Knowledge Sharing**: Shared understanding of system architecture and requirements
- **Collective Ownership**: Team responsibility for code quality and system reliability, and sometimes even shared features in development

4.3 Transparency

4.3.1 Open Development Process

Our transparency practices ensure accountability and continuous improvement:

Public Repository and Documentation:

- **Open Source Approach**: Public GitHub repository with comprehensive documentation
- **Transparent Project Management**: Public project board with visible progress tracking
- **Version Control History**: Complete development history with detailed commit messages

Regular Progress Reporting:

- **Demo Presentations**: Regular demonstrations of system capabilities and progress
- **Technical Documentation**: Architecture diagrams, deployment models, and system designs
- **User Manuals**: Complete user documentation and installation guides

4.3.2 Quality Metrics and Reporting

Transparent reporting of our engineering excellence:

Code Quality Metrics:

- **Test Coverage**: 80% minimum coverage tracked and reported
- **Code Quality**: ESLint and Prettier compliance across all code
- **Performance Metrics**: Performance testing with detailed reports
- **Security Assessments**: Comprehensive security testing with vulnerability reports

Project Transparency:

- **Public Project Board**: `https://github.com/orgs/COS301-SE-2025/projects/265/views/1`
- **Team Website**: `http://gititdone2025.site`
- **Comprehensive Documentation**: All project documentation publicly available
- **Release Management**: Automated releases with detailed change logs

5 Conclusion

TaxiTap by Git It Done exemplifies software engineering excellence through our comprehensive approach to building software, operational practices, and collaborative culture. Our commitment to coding standards, rigorous testing, and transparent development processes has resulted in a robust, scalable, and user-friendly platform that addresses real-world Taxi industry related challenges.

Through our structured approach to software development, comprehensive monitoring and data integrity practices, and collaborative team culture, we have demonstrated the principles of engineering excellence that make TaxiTap not just a functional application, but a showcase of professional software engineering practices.

Our project represents more than technical achievement—it embodies the values of quality, collaboration, and innovation that define software engineering excellence in the modern development landscape.